



6

Производительность



От чего зависит скорость выполнения программ?

- Системный и архитектурный аспект:
 - Архитектура (монолитная, n-tier, потоковая...).
 - Виртуализация и кластеризация.
- Низкоуровневый аппаратный аспект:
 - Частота, количество процессоров, количество ядер.
 - Объем и скорость кэшей, памяти, дисковой подсистемы.
- Программный аспект:
 - Используемые алгоритмы и структуры данных (пузырьковая сортировка vs Qsort).
 - Пулы, многопоточность и блокировки.
- Человеческий фактор (ошибки).

Рассмотрим аспекты современных систем, связанные с производительностью. В первую очередь это системный и архитектурный аспекты. Нужно учитывать, что при размещении программы в вычислительной среде, там уже работают другие программы. Кроме того, вычислительная среда построена согласно определённой архитектуре, которая задаёт структурные особенности выполняемых в ней программ, их параметры и характеристики. Такая архитектура может быть:

- Распределённой, где требуется баланс производительности на всех уровнях обработки информации (например, в классической трехзвенной архитектуре низкая производительность слоя БД может привести к падению производительности уровня бизнес-логики).
- Кластерной, где требуется балансировка между различными узлами кластера.
- Виртуализированной, где на одной серверной ферме работает много различных подсистем, взаимно влияя друг на друга из-за общих ограничений самой фермы.

Низкоуровневый аппаратный аспект связан с техническими характеристиками аппаратуры. Например, от тактовой частоты процессора зависит линейная скорость выполнения каждого потока программы, от размера кэшей и уровней конкуренции разных потоков зависит то, как много данных могут быть помещены в кэш второго уровня и насколько долго они могут там находиться, что также влияет на скорость обработки этих данных.

Программный аспект связан с выбором подходящих алгоритмов для разрабатываемых программ. Зачастую сроки сдачи в эксплуатацию являются более важным фактором, чем оптимальность решения поставленных задач. Примером программного аспекта является выбор между использованием метода пузырьковой сортировки и метода Qsort. Первый метод значительно менее производителен, чем второй, однако его реализация занимает меньший объем памяти. Кроме того, важными факторами могут являться пулы, реализация многопоточности, блокировки и др.

Важным фактором является человеческий — ошибки, неправильно принятые архитектурные решения, узкое знание предметной области и т.д.



Влияние средств наблюдения на результаты

- Анализ производительности — научный эксперимент:
 - Выбрать критерии оценки.
 - Выбрать средства измерения:
 - Неинтрузивные (non-intrusive) — не влияющие на результаты.
 - Слабоинтрузивные.
 - Интрузивные.
 - Выбрать нагрузку и/или нагрузить.
 - Провести анализ результатов.
 - Изменить один параметр/переписать один участок кода.
 - Повторять до полного удовлетворения.



Любая работа по анализу производительности похожа на научный эксперимент. Сначала выбираются критерии оценки — параметры, по которым можно оценить производительность программы. После этого выбираются средства измерения. Различаются *неинтрузивные* (non-intrusive) средства измерения, не влияющие на результаты, *слабоинтрузивные* (например, вольтметр вносит небольшие искажения в измеряемое напряжение) и *интрузивные* (например, весы у нечистого на руку продавца на рынке).

Применительно к измерению производительности вычислительных систем, практически любое измерение тем или иным образом влияет на результат самого измерения. Например, счётчики производительности операционной системы работают незаметно для пользователя (ибо они всё равно будут собираться ОС на системном уровне), но, если бы они не собирались, процессорное время на уровне системы не тратилось бы, и отдавалась пользовательской программе.

После выбора критериев и средств измерения, необходимо выбрать нагрузку и соответствующим образом нагрузить систему. Задача выбора нагрузки сложна тем, что эта нагрузка должна быть максимально приближена к реальной пользовательской нагрузке, т.е. её нужно сделать максимально *эквивалентной*. Во время работы программы под нагрузкой производится сбор результатов — например, системными утилитами производится периодический замер счётчиков ОС. Периодичность сбора и объём получаемых журналов, в свою очередь, тоже может влиять на результаты.

Затем проводится анализ результатов и исключение данных, наведённых средствами измерения. Например, если журнал измерений растёт со скоростью 1Мб/с, то это значение нужно исключить из производительности дисков.

После анализа выбирается один из параметров программы или участков кода, и варьируется/переписывается. Эксперимент повторяется. Потом аналогичным образом изменяется другой параметр и т.д.

Процедура повторяется до получения удовлетворительного результата производительности. На практике этот процесс происходит постоянно, как пели The Rolling Stones: "I can't get no satisfaction, 'Cause I try and I try and I try and I try".

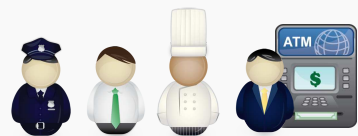


Основные характеристики и ключевые понятия

- Время отклика, время полного обслуживания и пропускная способность.
- Утилизация (%util или %busy) и (%wait) ожидание ресурса.
- Точка насыщения и масштабируемость.
- Эффективность:
 - CPU на 100% занят, это не значит, что система функционирует эффективно.
- Ускорение и прирост производительности.
- Java: Скорость запуска и memory footprints.








Параметры производительности тесно связаны между собой. Математически точные их определения будут представлены в курсе теории массового обслуживания.

Время отклика системы (latency) измеряется от выдачи запроса до получения первых результатов. Например, время отклика банкомата может быть определено как время с момента ввода карточки до получения первой порции денег, при этом *полное время обслуживания* определено до получения полной суммы (в несколько приёмов).

Пропускная способность показывает, какое максимальное количество запросов за единицу времени может пройти через канал ввода-вывода, систему, или её конкретный узел. Очевидно, что пропускная способность на верхнем рисунке выше, чем на нижнем.

Утилизация ресурса (%util) показывает, какую долю времени ресурс занят полезной работой. Если банкомату из десяти минут времени работы девять минут приходится обслуживать клиентов, то его утилизация составит 90%. Ожидание (%wait) ресурса учитывает среднюю длину очереди. % ожидания показывает сколько времени очередь *не пуста*.

В любых системах есть *точка насыщения* — момент, когда нагрузка достигает предельного значения, которое может обработать устройство или программа. Иными словами, точка насыщения — это точка достижения максимально возможной производительности. После достижения данной точки производительность может снижаться достаточно резко (обычно — экспоненциально). Более плавное снижение говорит о том, что основные составные компоненты системы хорошо сбалансированы. *Масштабируемость* характеризует то, насколько можно количественно расширить и нагрузить систему.

Эффективность ПО показывает соотношение полезной работы системы к общему количеству работы. Например, загрузка виртуальных страниц из swar-области заставляет систему загружать процессор и диск, но общая эффективность программы падает. *Ускорение работы и прирост производительности* показывают, насколько больше полезной работы выполняет программа после внесения изменений.

В Java "memory footprints" называют стратегии использования памяти. Если работа с памятью ведётся некорректно, то скорость её работы резко снижается.



Классический "нисходящий"

метод поиска узких мест



- Исключение ошибок аппаратуры и администратора:
 - Системные журналы, файлы конфигурации системы и прикладного ПО.
- Общесистемный и межузловой мониторинг:
 - CPU/память, подсистема IO, дисковые массивы, сеть, каналы передачи, операционная система, виртуализация...
- Мониторинг приложения:
 - Алгоритмические проблемы, проблемы API, многопоточность, блокировки, синхронизация.
- Мониторинг микроархитектуры:
 - Выравнивание данных, оптимизация кэшей, пузырьки конвейера, предсказание переходов.

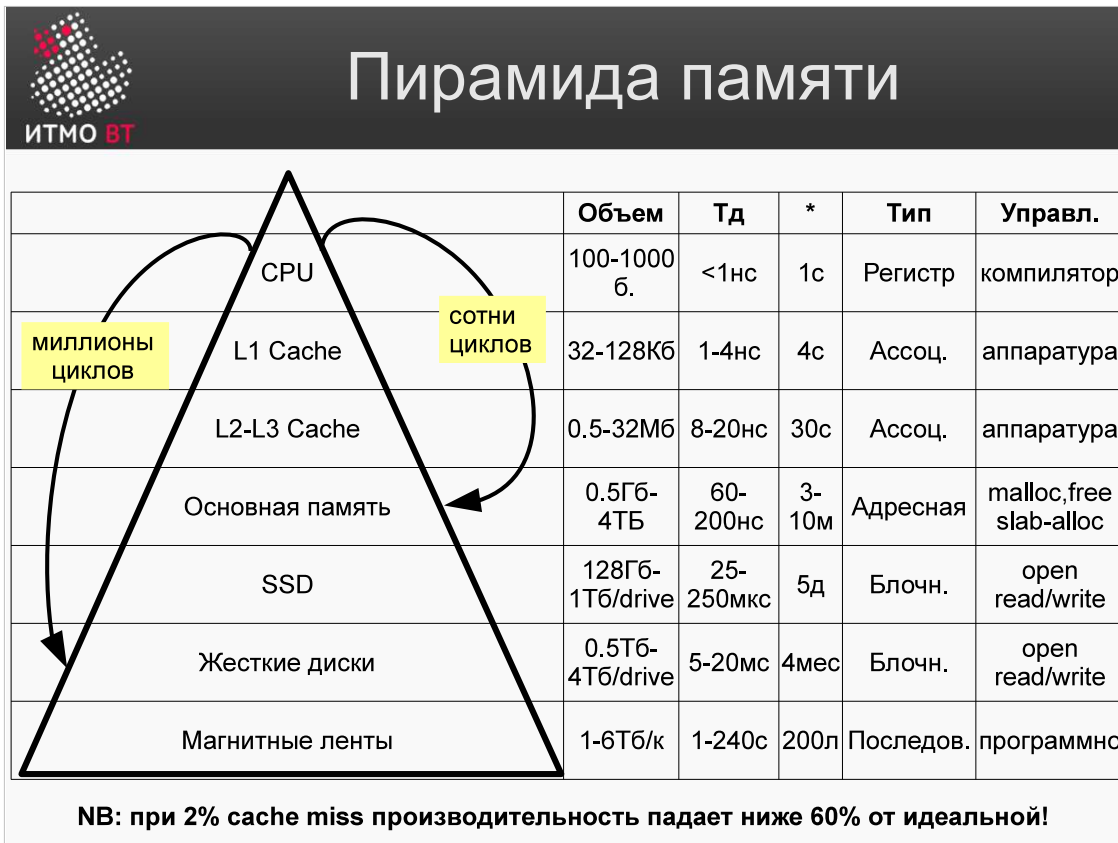
Классический "нисходящий" метод поиска узких мест последовательно рассматривает систему от более общих компонент к более частным.

В первую очередь администратор системы ищет ошибки аппаратуры и конфигурации системы. Проверяются системные журналы, файлы конфигурации системы и прикладного ПО. Сбойный блок на диске или дефект соединительного кабеля может привести к повторному чтению одного и того же участка информации, что снизит скорость выполнения операций ввода-вывода. В системных журналах такие ошибки видны сразу, при этом система обычно "честно" сообщает, к примеру, что из-за сбоев контрольной суммы скорость обмена по шине снижена. Администратор, например, может ошибочно выделить мало памяти для Java-машины, результатом чего будут частые сборки мусора, останавливающие систему. С другой стороны, выделение слишком большого объема памяти приведёт к избыточному накоплению мусора и длительной его очистке в перспективе, при этом основная задача, естественно, решаться не будет.

После исключения ошибок начинается собственно наблюдение за системой. Операционная система обладает достаточно развитыми средствами наблюдения за своими подсистемами. Средства системного, межузлового мониторинга и мониторинга виртуальных машин помогают наблюдать общую картину работы приложения и определить, на что в ней тратится основное время.

Следующим этапом является наблюдение за самим приложением, т.е. фиксируются и анализируются алгоритмические проблемы, проблемы API, проблемы, связанные с многопоточностью, блокировками и синхронизацией. В качестве средств наблюдения используются средства мониторинга и профилирования приложений.

Если полученного в ходе предыдущего этапа прироста производительности оказывается недостаточно, происходит переход к мониторингу микроархитектуры: проверяются количество инструкций, обрабатываемых процессором за один такт, выравнивание данных, оптимизация кэшей, пузырьки конвейера, предсказание переходов и т.д. Для улучшения ПО на этом уровне необходимо знать ассемблерный код, архитектуру процессора, принципы компиляции и пр. Обычно в пользовательском ПО до этого уровня не доходят из-за сложности внесения изменений.



При разработке программ учёт скорости доступа к различным компонентам архитектуры вычислительной среды может существенно повлиять на итоговую производительность.

Сверху пирамиды находится самая дорогая память с наименьшим объёмом. Внизу находится самая дешевая память с максимальным объёмом. Чем ниже память в пирамиде, тем дороже в плане затрат времени обходится обращение к ней. Например, обращение к основной памяти (в середине пирамиды) может занимать в сотни раз больше времени, чем обращение к регистрам процессора или к кэшу первого уровня. Поэтому минимизация количества обращений к нижним уровням пирамиды во много раз повышает производительность — иногда в сотни или даже миллионы раз. Если исключить обращения к жёсткому диску и однократно записывать необходимые данные в память, то производительность может увеличиться в миллионы раз.

Для иллюстрации такого повышения скорости работы программы рассмотрим простой пример. Если отождествить такт процессора со скоростью диалога в процессе межличностного общения (колонка «*» на слайде), то на вопрос, заданный одним собеседником (например, "Как дела?"), ответ, данный собеседником, придёт через 1 секунду (если он отвечает со скоростью процессора). Если данные, необходимые для ответа, содержится в кэш-памяти первого уровня, то время ответа будет всё ещё приемлемым, всего 4 секунды. Если в кэше второго уровня — то уже 30 секунд, что для беседы выглядит куда как странно. Можно подумать, что человек думает о чём-то другом. Оперативная память даст задержку в минуты, что уже совсем неприемлемо для беседы. Он что, собирается соврать?!

Обращение к постоянной памяти, SSD и жестким дискам даст задержку в дни или месяцы, что в сотни тысяч и миллионы раз замедлит быстроедействие процессора. Получение архивной копии данных (аналог постоянной памяти человека, заложенной в детстве) займёт и вовсе сотни лет.

Учёт архитектуры ЭВМ позволяет сделать разрабатываемые программы существенно (на несколько порядков) быстрее.



Системный мониторинг производительности

- Собирает параметры операционной системы и программ:
 - CPU: user%, system%, idle%, ctxt_sw, interrupts, load average.
 - IO: b, kb, mb, blk read & write/s, op/s, wait time.
 - VM: free, buff, cache, pagein/s, pageout/s, scan time.
 - Network : up (TX) & down (RX) streams, errors, collisons.
 - Детальные данные других подсистем.
- Обычно неинтрузивен (счётчики и так собираются), хотя возможны варианты.

Системный мониторинг производительности осуществляется при помощи средств ОС, позволяющих получать и анализировать различные параметры операционной системы и запущенных под её управлением программ.

Для мониторинга CPU важно знать количество процессов на уровне пользователя, сколько ресурсов CPU система тратит на эти процессы, время простоя процессоров, параметры ctxt_sw (context switch — переключение контекста), interrupts (прерывания), и load average (средняя загрузка очереди исполнения).

Для мониторинга эффективности работы подсистемы ввода-вывода важно знание числа байтов, килобайтов, мегабайтов (и, иногда, блоков), читаемых и записываемых за секунду, и за какое количество операций это происходит (например, если число операций примерно равно числу переданных байтов, это может указывать на неэффективность алгоритма работы с памятью, слишком маленький размер буфера обмена данными и отсутствие кэширования). Параметр wait time применительно к задачам ввода-вывода определяет время ожидания до получения первых данных.

При мониторинге подсистемы виртуальной памяти отслеживается количество немедленно готовой к использованию свободной памяти (free), объём занятых буферов (buff) и кэшей (cache), количество страниц, читаемых (pagein/s) и записываемых (pageout/s) системой в область подкачки, и scan time — параметр, определяющий скорость сканирования изменённых страниц из этой памяти.

При решении задачи мониторинга утилизации сети отслеживаются потоки upstream (TX, передаваемые данные) и downstream (RX, получаемые данные). Также учитываются возникающие ошибки и коллизии (последние являются редкостью в современных системах, "умные" коммутаторы избавляют от коллизий, передавая данные между всеми сетевыми портами одновременно).

ОС также может предоставить детальные данные других подсистем (например, централизованную статистику ядра).

Обычно такой системный мониторинг неинтрузивен и не оказывает влияния на работу программы, так как ядро всё равно собирает указанные выше счетчики, и их нужно просто прочитать из системы.

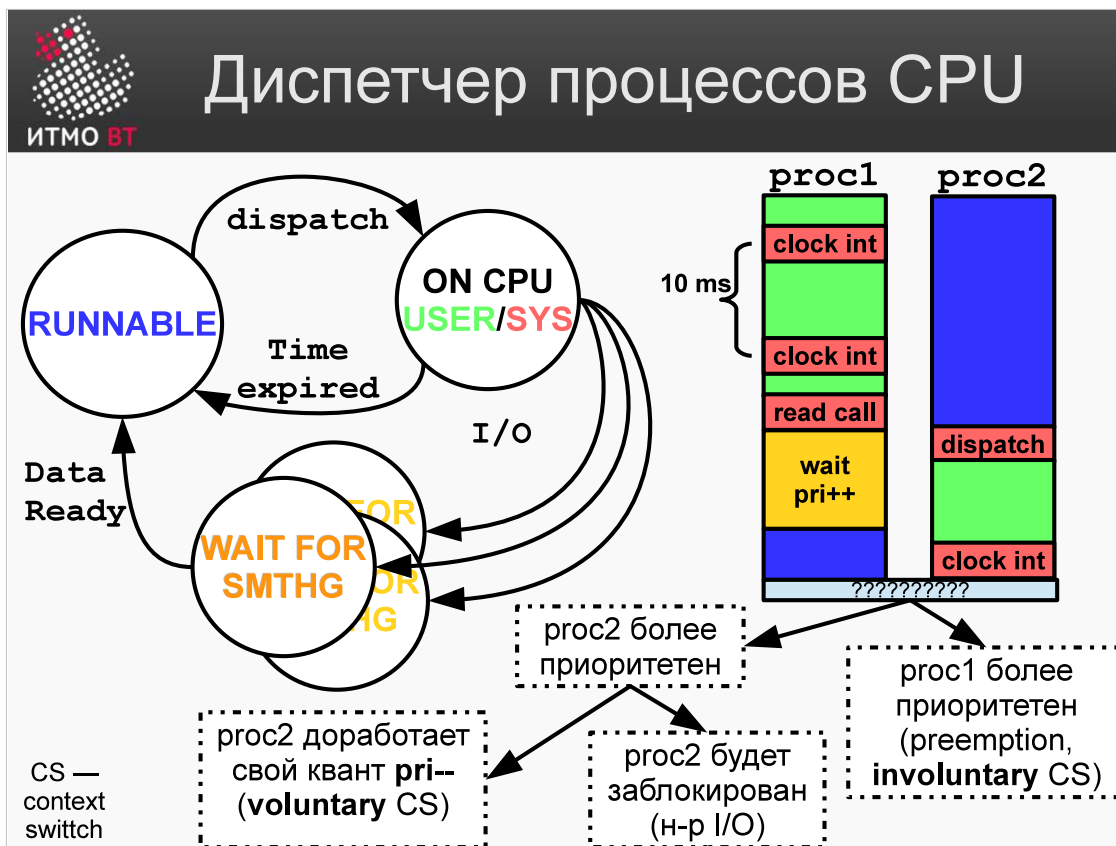


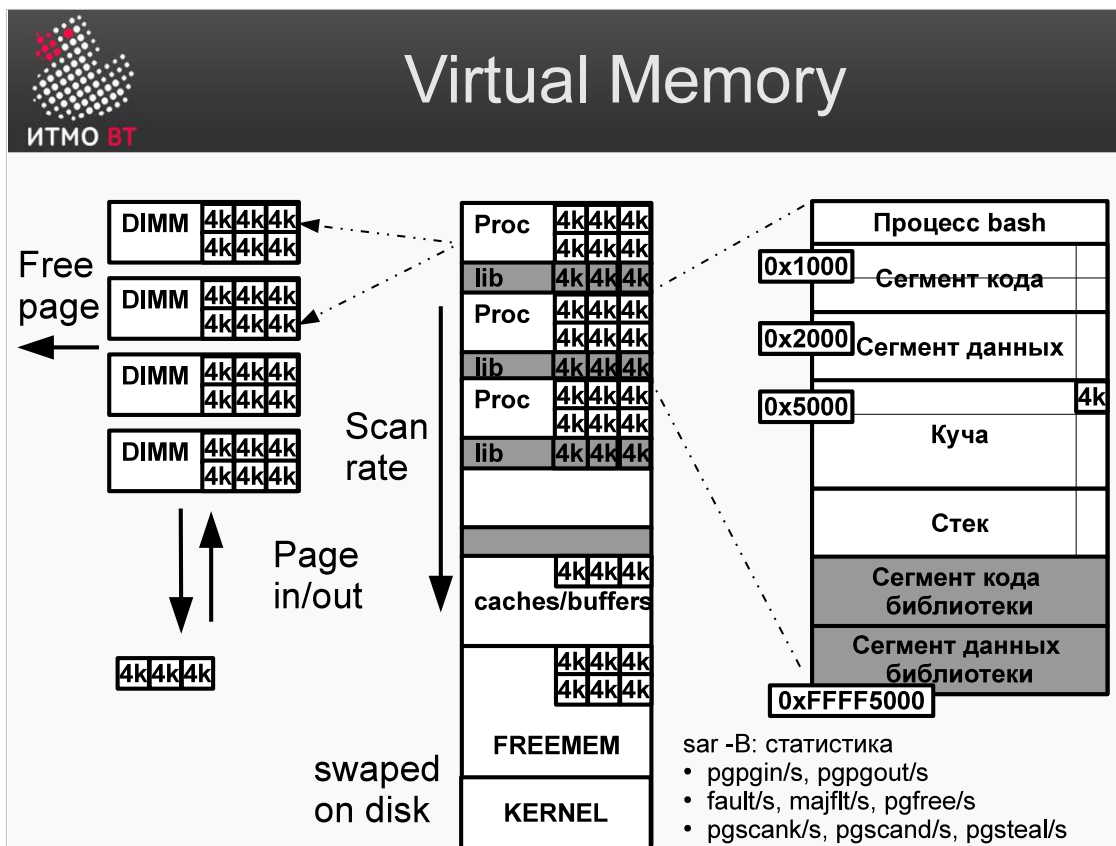
Схема диспетчеризации процессов CPU в сильно упрощенном виде показана на слайде. Показанная схема приблизительно одинакова во всех современных операционных системах. Результаты диспетчеризации можно контролировать при помощи системных утилит.

Процесс или поток внутри процесса может глобально находиться в трёх состояниях: он может быть готов к исполнению (Runnable), находиться на исполняющем устройстве (либо на уровне пользователя), либо на уровне ОС (On CPU User/Sys), или ожидать ввода/вывода, освобождения блокировки и т.п. (Wait for smthg). При этом, во время ожидания нет необходимости занимать ресурсы процессора.

Пусть процесс **proc1** в настоящий момент находится на процессоре и работает программа пользователя. Раз в 10 мс (время может быть другим) происходит т.н. прерывание от часов (clock interrupt), во время которого ОС наращивает счетчики производительности, проверяет наличие в очереди на исполнение процесса с более высоким приоритетом, чем у текущего, а также проверяет, не исчерпал ли текущий процесс свой квант времени. В нашем случае, исполнение **proc1** продолжается, так как квант времени не исчерпан и **proc2** не является более приоритетным, чем **proc1**, поэтому он остается в режиме готовности в очереди на выполнение. Так продолжается до тех пор, пока не будет исчерпан квант времени, либо не будет сделан вызов из **proc1** к устройству ввода-вывода.

При появлении такого вызова (на слайде – **read call**), **proc1** переходит в режим ожидания, и происходит т.н. context switch (процессор переходит к исполнению другого процесса, сохранив окружение предыдущего). Внутри ядра происходит диспетчеризация, в ходе которой выбирается процесс с высшим приоритетом, готовый к выполнению. Процесс **proc2** начинает выполняться. Процесс **proc1** отправляется в состояние ожидания, но у него повышается приоритет (чем больше время ожидания, тем больше увеличивается приоритет, и наоборот, трата процессорного времени приводит к понижению приоритета, что типично для систем разделения времени).

В момент прерывания возможны различные варианты действий. Если приоритет **proc2** выше, чем у **proc1**, то он либо дорабатывает свой квант времени, либо выполняет операцию, связанную с вводом-выводом. Если **proc1** более приоритетен, то он снова перейдет на процессор (произойдет вытеснение – **preemption**, или несвободное переключение контекста – **involuntary CS**).

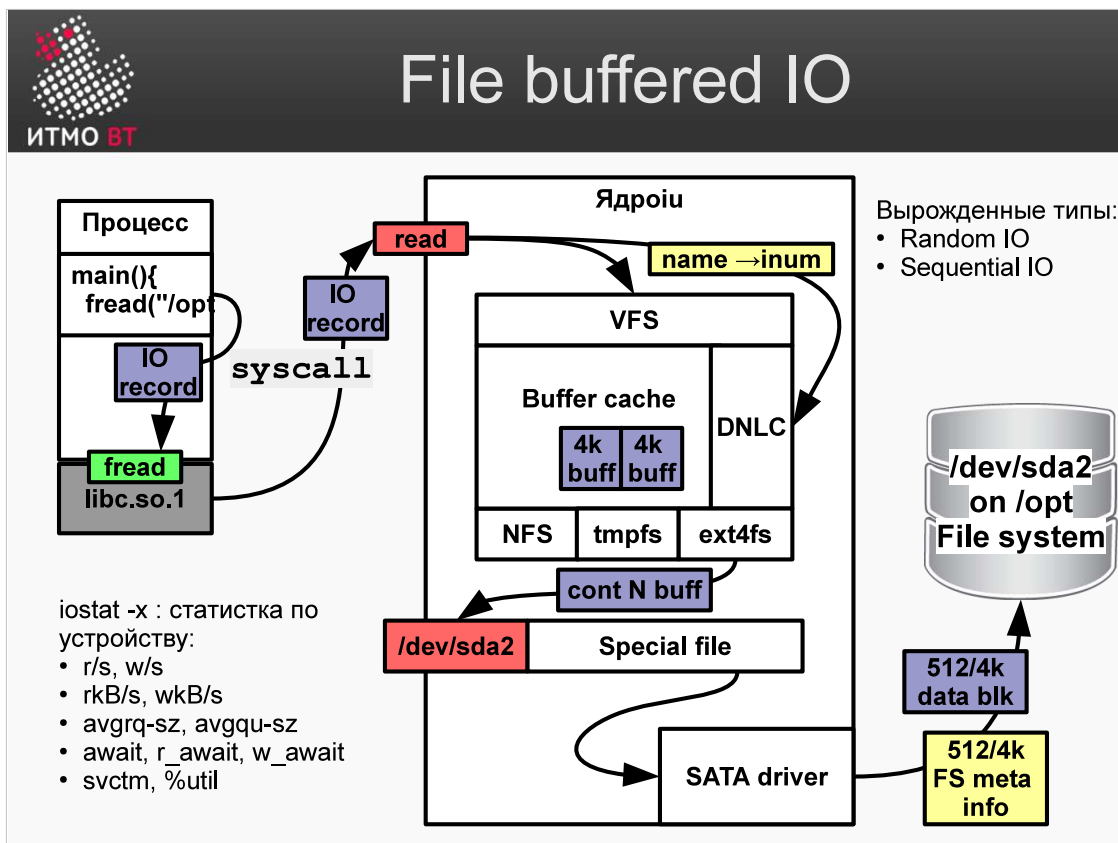


Виртуальная память — метод управления памятью компьютера, позволяющий выполнять программы, требующие больше оперативной памяти, чем имеется в компьютере, через автоматическое перемещение частей программы между основной памятью и вторичным хранилищем (например, жёстким диском).

В работе виртуальной памяти задействованы собственно физическая память, устройство подкачки на дисках (swar-устройство), и виртуальные страницы, которые могут принадлежать либо физической памяти, либо виртуальной памяти. Сегментно-страничная структура процессов типична для современных ОС, где каждая страница может находиться в нескольких состояниях: быть в памяти, находиться на swar-устройстве, или быть зарезервированной (необходимой для выполнения, но физически еще нигде не расположенной). Вся память процесса поделена на страницы (которые могут быть размером 4Кб и больше, например, 2Мб). Размер страницы влияет на количество *мэтингов* (записей отображений структур виртуальных адресов ОС на страницы внутри банка памяти). Следует отметить, что используемые для чтения страницы (такие, как код программы или статически скомпилированные данные) необязательно перемещать на swar-устройство, их можно просто удалить из памяти, потому, что они могут быть загружены из исполняемого файла программы. Страницы, связанные с файлами, обычно называют *именованной* памятью, а те, которые созданы динамически в куче (*heap*), получили название *анонимные*.

С виртуальной памятью связано несколько параметров. Scan rate — число просканированных страниц за единицу времени. Если значение scan rate высоко, это означает недостаток оперативной памяти или постоянную работу ОС, связанную с манипуляцией страницами. ОС при недостатке памяти производит последовательное сканирование редко используемых, неизменных страниц, и принимает решения (в зависимости от их типа) о записи их на диск или удалении. Если процессор обратился к отсутствующей в памяти странице, происходит *страничный сбой* (page fault). Данное событие может быть минорным (minor), когда для страницы нужно просто создать запись (мэпинг) в таблицах адресации, или *существенным* (major), когда страница должна быть загружена из диска подкачки.

Сбор статистики виртуальной памяти в ОС unix/linux осуществляется при помощи команды sar -B. Аргументы этой команды и собираемые данные представлены на слайде.



На слайде приведена организация буферизованного ввода-вывода. При чтении данных вначале указывается количество байт, которые нужно прочитать (IO record size). Интенсивность чтения данных определяется требованиями программы.

Предположим, внутри программы производится обращение к функции `fread`, которая находится в системной библиотеке. Функция `fread` формирует запрос к ядру и вызывает соответствующую функцию ядра `read`. Ввод-вывод в ядре попадает в подсистему VFS (virtual file system, виртуальная файловая система), и запрос попадает на уровень, не связанный с конкретной файловой системой. Имя файла преобразуется в DNLC (directory name lookup cache — кэш, ускоряющий обработку имен файлов) в номер файла inode в необходимой файловой системе.

ОС экономит ресурсы при обращении к устройствам ввода-вывода, и операция `read` будет работать в первую очередь с буферным кэшем (Buffer cache). В нём данные содержатся в виде блоков данных файла в ОЗУ, формируя промежуточное хранилище. Каждые 30 секунд (время зависит от ОС) данные, которые были помечены как изменённые, сохраняются на диск. Ниже, под виртуальной файловой системой существуют реализации модулей ядра физических файловых систем. К точке монтирования подключается устройство с использованием его специального файла, после этого работу ведёт драйвер, и на уровне аппаратного интерфейса данные перемещаются в дисковое устройство.

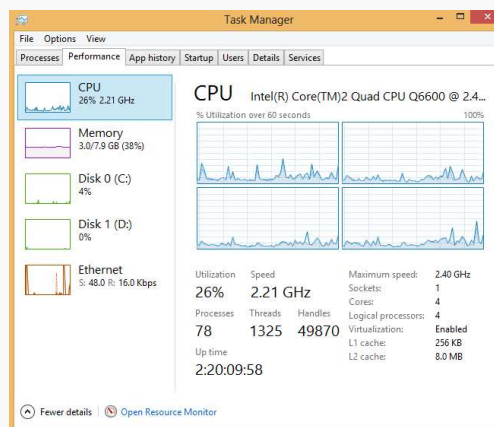
Для каждой файловой системы существуют наблюдаемые параметры: количество чтений (r/s), количество записей (w/s), объем читаемых и записываемых данных (rkB/s и wkB/s), средние времена запросов (avgrq-sz и avgqu-sz), время ожидания (await, r_await, w_await), время обслуживания (svctm), и процент занятости устройства (%util).

Применительно к вырожденным типам обмена с дисковой подсистемой различают *случайный доступ* (random IO, каждый запрос обращается к новому месту диска) и *последовательный доступ* (sequential IO, данные записываются или читаются большими группами последовательно), при этом скорость чтения/записи во втором случае во много раз выше, чем в первом. Для случайного доступа существенно повысить скорость обмена данными может использование твердотельных накопителей (solid-state drive, SSD) из-за отсутствия в них (в отличие от дисков) движущихся частей. Команда `iostat` показывает рассмотренные выше характеристики обмена.



Мониторинг Windows

- Встроенные:
 - Task Manager.
 - Resource Monitor & Performance Monitor.
 - Reliability Monitor.
 - **Microsoft SysInternals.**
- Коммерческие и свободно распространяемые:
 - <https://blog.serverdensity.com/windows-monitoring-tools/>

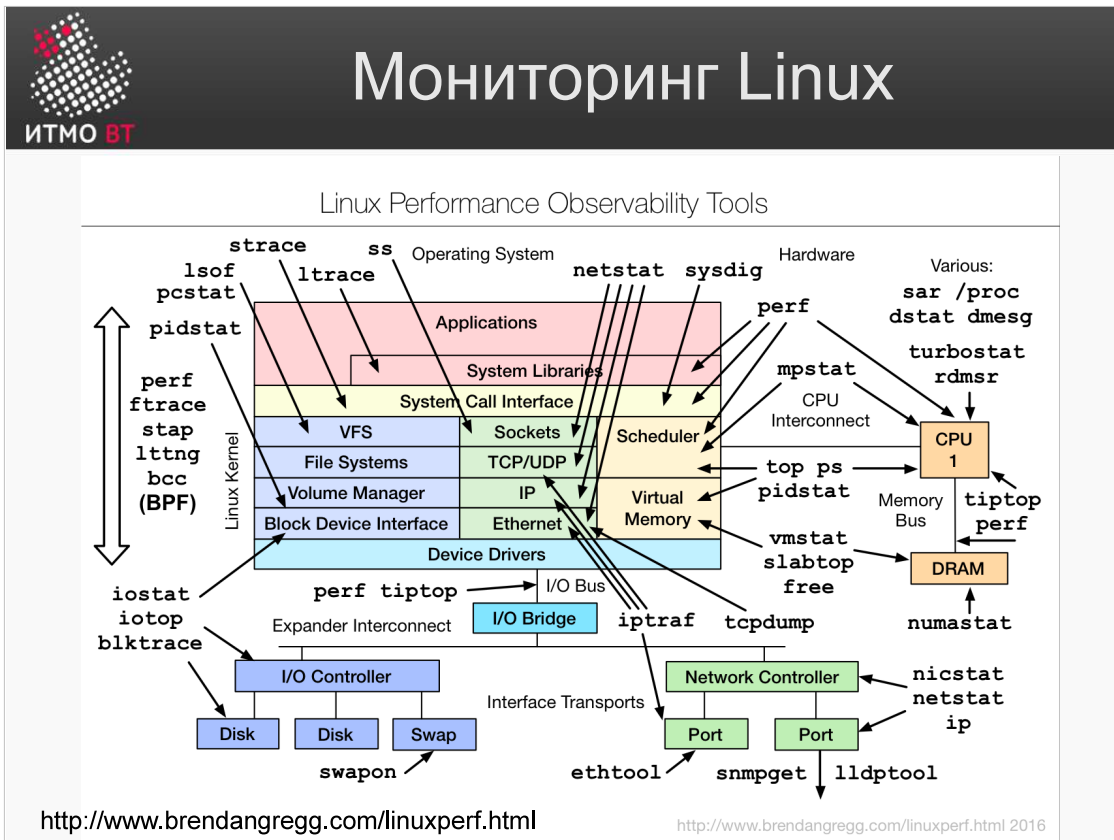


В различных операционных системах существуют разные средства мониторинга.

В Windows самым распространённым является стандартное средство системного мониторинга Task Manager, где, помимо всего прочего, выводится статистика использования процессора, памяти, дисковой подсистемы и др.

Для более детального исследования поставляются также специальные системные оснастки, такие, как Resource Monitor & Performance Monitor, Reliability Monitor и т.д.

Наиболее детальную информацию можно получить из средства Microsoft SysInternals (независимая разработка, впоследствии купленная компанией), которое базируется на внутрисистемных счетчиках и информации, вследствие чего показывает наиболее точные результаты по сравнению с другими средствами.



В ОС семейства Linux ситуация на первый взгляд выглядит сложнее, чем с Windows. В Linux существует большое число средств мониторинга, наблюдающих за определёнными подсистемами ОС и учитывающих особенности архитектуры этих подсистем. В общем случае они являются неинтрузивными.

На слайде видно, из счётчиков каких подсистем такие стандартные для всего семейства Unix средства, как vmstat, mpstat, iostat и netstat берут информацию о производительности.

Утилита top позволяет динамически наблюдать характеристики запущенных процессов, такие, как текущий приоритет (PR), занимаемая память (VIRT — общий размер адресного пространства процесса, RES — размер в физической памяти, SHR — в совместно используемой с другими процессами) и др. В статусной строке можно вывести большое число общесистемных характеристик.

Утилита sar имеет множество опций для вывода той, или иной системной информации ядра. Это одна из первых утилит мониторинга производительности во всём семействе ОС UNIX. В современной ситуации следует ее использовать осторожно, поскольку используемые в этой утилите подходы ко сбору некоторых метрик устарели.

Отдельно следует выделить средство perf, которое может собирать и показывать большое число характеристик не только ядра, но и запущенной под управлением perf программы. Perf умеет работать со счётчиками производительности процессора для сбора таких событий, как промахи мимо кэша.

Для детального наблюдения за процессом можно использовать strace. Эта утилита позволяет проводить трассировку системных вызовов, которые процесс выполняет к ядру ОС и системным библиотекам (таким, как libc.so). Strace в общем случае достаточно интрузивен.

System tap (stap) позволяет установить точки сбора информации в ядре, собрать и агрегировать информацию о подсистемах ядра. Это средство использует свой собственный язык программирования. В Solaris существует аналог stap, который называется dtrace, который впервые продемонстрировал подобный (слабоинтрузивный) подход к мониторингу.



Примеры: что тут происходит?

```

$ vmstat 1 5
procs -----memory----- ---swap-- ---io--- --system-- -----cpu-----
 r b  swpd  free  buff  cache   si   so    bi   bo    in  cs us sy id wa st
 6 1      0 302380 16356 271852    0    0   561   568   80  590 43  7 43  7  0
 1 0      0 300892 16364 273256    0    0    0    52   79  274 97  3  0  0  0
 2 0      0 299544 16364 274532    0    0    0    0   78  372 97  3  0  0  0
 1 0      0 298292 16368 275780    0    0    0    0   53  255 97  3  0  0  0
 1 0      0 296820 16368 277192    0    0    0    0   77  377 97  3  0  0  0

$vmstat 1 5
procs -----memory----- ---swap-- ---io--- --system-- -----cpu-----
 r b  swpd  free  buff  cache   si   so    bi   bo    in  cs us sy id wa st
 2 0 402944 54000 161912 745324    5   14   54   59  221  867 13  3 82  2  0
 1 0 402944 53232 161916 748396    0    0    0    0   30  213  3 97  0  0  0
 1 0 402944 49752 161920 751452    0    0    0    0   28  290  4 96  0  0  0
 1 0 402944 45804 161924 755564    0    0    0    0   29  188  2 98  0  0  0
 1 0 402944 42568 161936 758608    0    0    0 17456  272  509  7 93  0  0  0

$ vmstat 1 5
procs -----memory----- ---swap-- ---io--- --system-- -----cpu-----
 r b  swpd  free  buff  cache   si   so    bi   bo    in  cs us sy id wa st
 3 1 244208 10312 1552 62636    4   23   98  249   44  304 28  3 68  1  0
 0 2 244920 6852 1844 67284    0 544 5248  544  236 1655  4  6  0 90  0
 1 2 256556 7468 1892 69356    0 3404 6048 3448  290 2604  5 12  0 83  0
 0 2 263832 8416 1952 71028    0 3788 2792 3788  140 2926 12 14  0 74  0
 0 3 274492 7704 1964 73064    0 4444 2812 5840  295 4201  8 22  0 69  0

```

https://www.thomas-krenn.com/en/wiki/Linux_Performance_Measurements_using_vmstat

На слайде приведены примеры наблюдения за производительностью с помощью Linux-утилиты vmstat. Формат команды определяет, что нужно произвести 5 последовательных измерений с интервалом в 1 секунду.

Первая строчка под заголовком таблицы vmstat показывает усреднённые системные значения с момента запуска ОС, а вторая и последующие — мгновенные значения с последнего измерения.

В первом примере процессор загружен на 100%, и большая часть этой загрузки приходится на работу с приложением пользователя (столбец `cpu us`).

Во втором примере основное время процессор проводит на уровне ядра операционной системы (столбец `cpu sy`). Такие высокие значения нетипичны для нормальной работы системы. Можно предположить, что в каком-то модуле ядра дефект. Искусственно получить такую ситуацию можно чтением из датчика псевдослучайных чисел (который реализован в драйвере ядра) большими блоками, например при помощи команды: `dd if=/dev/urandom of=/dev/null bs=1024k`

В третьем примере значительная часть времени процессора приходится на ожидание на блокировках (столбец `cpu sy`). Наличие значительного дискового ввода-вывода (`bi`, `bo`) и высокая выгрузка на swar-устройство страниц памяти (`so`) указывают на ситуацию, когда недостаточно оперативной памяти. Этот недостаток можно также наблюдать в столбце `free`. В этом случае понятно, на каких блокировках ждёт процессор — это очередь записи на диск подкачки.



Примеры: что тут происходит?

(2)

```

$ vmstat 1 5
procs -----memory----- --swap-- --io-- --system-- -----cpu-----
 r b  swpd   free   buff  cache   si   so    bi    bo    in   cs us sy id wa st
 3 1 465872 36132 82588 1018364    7   17    70   127  214  838 12  3 82  3  0
 0 1 465872 33796 82620 1021820    0    0 34592    0  357  781  6 10  0 84  0
 0 1 465872 36100 82656 1019660    0    0 34340    0  358  723  5  9  0 86  0
 0 1 465872 35744 82688 1020416    0    0 33312    0  345  892  8 11  0 81  0
 0 1 465872 35716 82572 1020948    0    0 34592    0  358  738  7  8  0 85  0

$ vmstat 1 5
procs -----memory----- --swap-- --io-- --system-- -----cpu-----
 r b  swpd   free   buff  cache   si   so    bi    bo    in   cs us sy id wa st
 1 1      0 35628 14700 1239164    0    0   1740   652  117  601 11  4 66 20  0
 0 1      0 34852 14896 1239788    0    0      0 23096  300  573  3 16  0 81  0
 0 1      0 32780 15080 1241304    0    0      4 21000  344  526  1 13  0 86  0
 0 1      0 36512 15244 1237256    0    0      0 19952  276  394  1 12  0 87  0
 0 1      0 35688 15412 1237180    0    0      0 18904  285  465  1 13  0 86  0

$ vmstat 1 5
procs -----memory----- --swap-- --io-- --system-- -----cpu-----
 r b  swpd   free   buff  cache   si   so    bi    bo    in   cs us sy id wa st
 2 1 403256 602848 17836 400356    5   15    50    50  207  861 13  3 83  1  0
 1 0 403256 601568 18892 400496    0    0   1048   364  337 1903  5  7  0 88  0
 0 1 403256 600816 19640 400568    0    0    978    0  259 1142  6  4  0 90  0
 0 1 403256 600300 20116 400800    0    0    946    0  196  630  8  5  0 87  0
 0 1 403256 599328 20792 400792    0    0   1076    0  278 1401  7  5  0 88  0

```

Примеры данного слайда посвящены вырожденным случаям обмена с обычными (не-SSD) дисками. В таких дисках существуют несколько носителей информации ("блинов"), сам диск при этом разбит на дорожки, а те, в свою очередь, на сектора. Размер такого сектора составляет 512 байт или 4 Кб. Размер блока (используемого в многих системных утилитах, в т.ч. vmstat) в ОС Linux составляет 1024 байта, т.е. можно считать, что данные ввода-вывода представлены в килобайтах в секунду.

Чтобы произвести обмен с диском, необходимо установить читающую головку на нужную дорожку, дождаться поворота диска к нужному сектору и передать (прочитать или записать) информацию. Ядро при этом ожидает на блокировке, что видно во всех примерах на слайде — высокое время ожидания процессора (cpu wa).

В первом примере происходит активное чтение с диска (bi). Скорость этого чтения составляет 34 Мб/с., и является типичным для *последовательного* (sequential) чтения большого объёма данных с дисков из недалекого прошлого. К сожалению, на слайде нет информации по количеству операций ввода-вывода (информацию по этим операциям можно посмотреть в vmstat -d или iostat). Если провести анализ количества операций, то выяснится, что данные читались большими порциями (размерами блока ввода-вывода). Данные при таком способе выбираются с диска последовательно, количество лишних движений головки диска и его поворотов сведено к минимуму.

Характерное поведение для последовательной записи представлено во втором примере. При этом скорость записи (bo около 20 Мб/с) меньше скорости чтения. Это тоже типичное значение для скорости записи таких дисков.

В третьем примере показана картина *случайного* (random) чтения. При этом данные сильно разбросаны по диску и ввод-вывод происходит небольшими блоками. Это приводит к большим передвижениям головки по диску, скорость чтения падает до 1 Мб/с. Такую деградацию кардинально исправляют SSD. Третий пример также может быть примером работы с медленным устройством, таким, как cdrom.

Скорость чтения-записи жёсткого диска в его начале и конце может отличаться до 40% (в начале быстрее). Это связано с разницей линейной скорости движения головки над поверхностью в зависимости от угловой скорости вращения. Именно поэтому swar-область размещают в начале диска.



Системный анализ "за 60 секунд"

- uptime — load average за 1, 5, 15 минут.
- dmesg | tail — последние ошибки.
- vmstat 1 — есть ли свободная память, paging, распределение CPU.
- mpstat -P ALL 1 — распределение по CPU.
- pidstat 1 — статистика по процессам, горячие процессы .
- iostat -xz 1 — параметры ввода-вывода.
- free -m — проверка исчерпания кэшей/буферов.
- sar -n DEV 1 — сетевая статистика по интерфейсам.
- sar -n TCP,ETCP 1 — сетевая статистика по соединениям.
- top — онлайн-мониторинг параметров.



http://www.brendangregg.com/Articles/Netflix_Linux_Perf_Analysis_60s.pdf

На слайде приведён практический подход, позволяющий провести системный анализ «за 60 секунд». Это последовательность команд, позволяющих быстро оценить ситуацию и составить первое впечатление о возможном источнике ошибок.

В первую очередь, команда uptime позволяет показать load average — среднюю загрузку процессорной подсистемы. Значения load average показывают количество готовых к выполнению процессов в очереди на диспетчеризацию за 1, 5 и 15 минут. Если таких процессов много, то система не справляется с нагрузкой.

После этого просматриваются последние ошибки с помощью dmesg | tail. Затем проверяется виртуальная память (с помощью vmstat 1) и распределение процессов по CPU (с помощью mpstat -P ALL 1), так как возможна ситуация, когда один процессор занят, а остальные простаивают. Проверяются наиболее «горячие» процессы (с помощью pidstat 1), характеристики ввода-вывода (с помощью iostat -xz 1), память (free -m — проверка исчерпания кэшей/буферов) и сетевая статистика по интерфейсам и по соединениям (sar -n DEV 1 и sar -n TCP,ETCP 1).

Практически все эти характеристики вместе можно проверить с помощью команды top (онлайн-мониторинг параметров), которая представляет собой «швейцарский нож» для администратора.



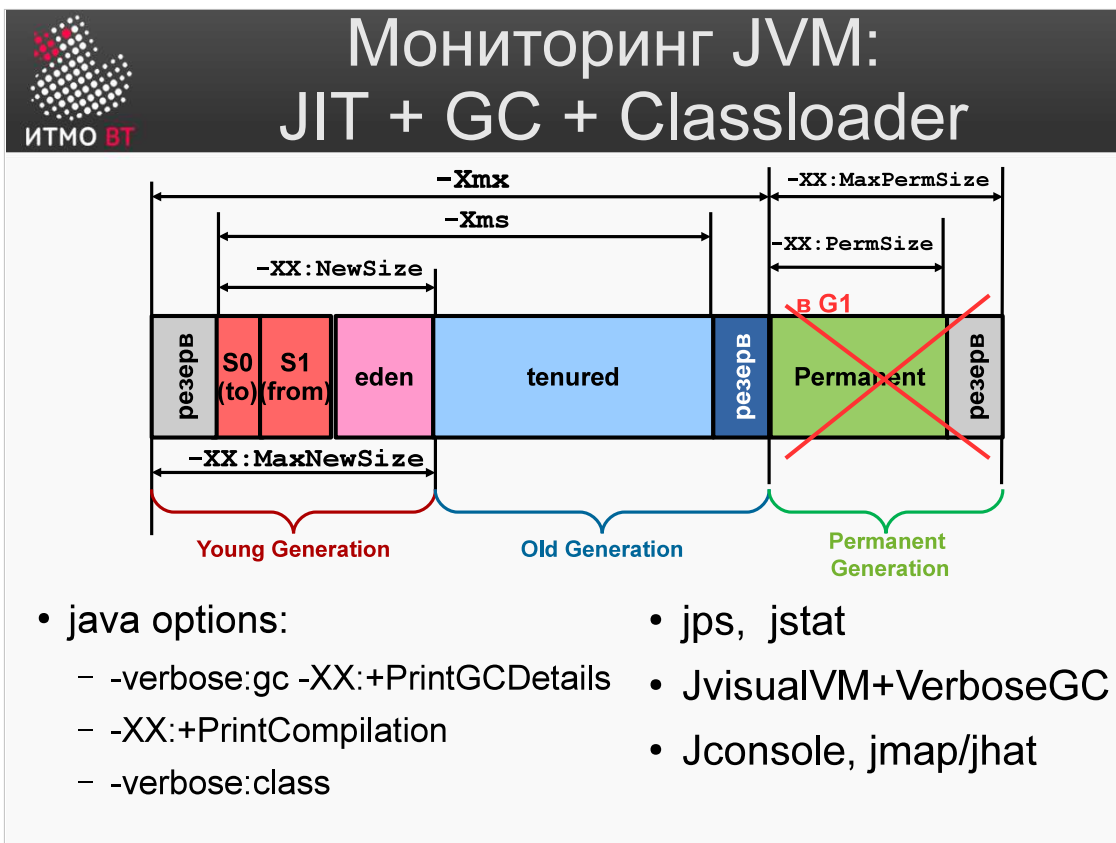
Создание тестовой системы и нагрузчики

- Иногда запрещено наблюдать боевую систему:
 - Средства мониторинга вносят искажения в нормальную работу.
 - Страх наличия дефектов в средствах мониторинга.
- Необходимо создать тестовую систему и нагрузить ее "так же, как и основную":
 - Средства создания синтетической нагрузки (см. тестирование).
 - Средства записи реальной нагрузки.

В корпоративных системах наблюдение реальных, критически важных для бизнеса систем часто запрещено из-за страха перед внесением искажений в нормальную работу системы средствами мониторинга, или перед наличием дефектов в этих средствах. В таких случаях обычно создают отдельную *тестовую* систему, являющуюся полной копией реальной, и проводят измерения на ней.

Для тестовой системы нужно иметь возможность создавать нагрузку, близкую по характеристикам к реальной пользовательской нагрузке. В тестовой системе также возможно использование интрузивных средств мониторинга.

Нагрузку, аналогичную реальной, можно создать с помощью средства создания синтетической нагрузки или средства записи реальной нагрузки, позволяющего запомнить нагрузку на реальном устройстве и использовать эти данные для тестовой системы. При этом синтетическая нагрузка всегда будет отличаться от реальной, и в средствах создания такой нагрузки используется большое число параметров, позволяющих гибко её настроить.



Память, используемая JVM (виртуальная машина Java), состоит из областей памяти для молодого поколения (Young Generation), выживших объектов (Old Generation) и области постоянного хранения (Permanent Generation). В современных JDK Permanent Generation отсутствует.

Создаваемые объекты помещаются в eden. Через определённое время запускается *минорная* сборка мусора (Minor Garbage Collection), и объекты, на которые до сих пор имеются ссылки, копируются в S0 или S1 — области в молодом поколении для выживших (перенесших сборку мусора) объектов. Использование S0 и S1 происходит по очереди. При следующей сборке мусора на наличие ссылок проверяются объекты и в eden, и, предположим, в S0. Если на объект в S0 имеются ссылки, то он перемещается в S1. Так продолжается некоторое количество сборок мусора, и, если объект пережил их все (число может динамически варьироваться от 8 до 32), то он помещается в область выживших объектов. В перманентном поколении обычно находятся объекты, которые специально туда помещены (загруженные классы, пулы строк и т.п.).

Параметрами JVM можно задать размеры памяти для всех поколений. Ключи для задания параметров представлены на слайде. Зарезервированные области выделены операционной системой согласно ключам запуска, но ещё не используются JVM, так как в этом нет пока необходимости.

С точки зрения Java-машины возможно наблюдать всего за тремя группами характеристик: JIT (just-in-time) компиляцией, сборкой мусора, и загрузкой классов Classloader'ом. Для Java существуют развитые средства мониторинга, такие, как jps, jstat, jvisualVM+VerboseGC (удобное средство для мониторинга сборщика мусора), JConsole, jmap/jhat и т.д.

Загрузчик классов сильно влияет на время запуска приложения при большом количестве подгружаемых библиотек. Можно уменьшить время, затрачиваемое на запуск путём предварительной компиляции необходимых классов и помещения их в отдельный файл для быстрой загрузки.



Профилировщики приложений

- Нужны для анализа:
 - Времени исполнения функции/метода.
 - Создания объектов в памяти.
 - Поток / соревнования за блокировку.
- Два основных подхода:
 - Внедрение диагностических точек в указанный набор функций (AAAA! только не во ВСЕ!!!)
 - Периодические прерывания и сбор информации, затем сортировка.

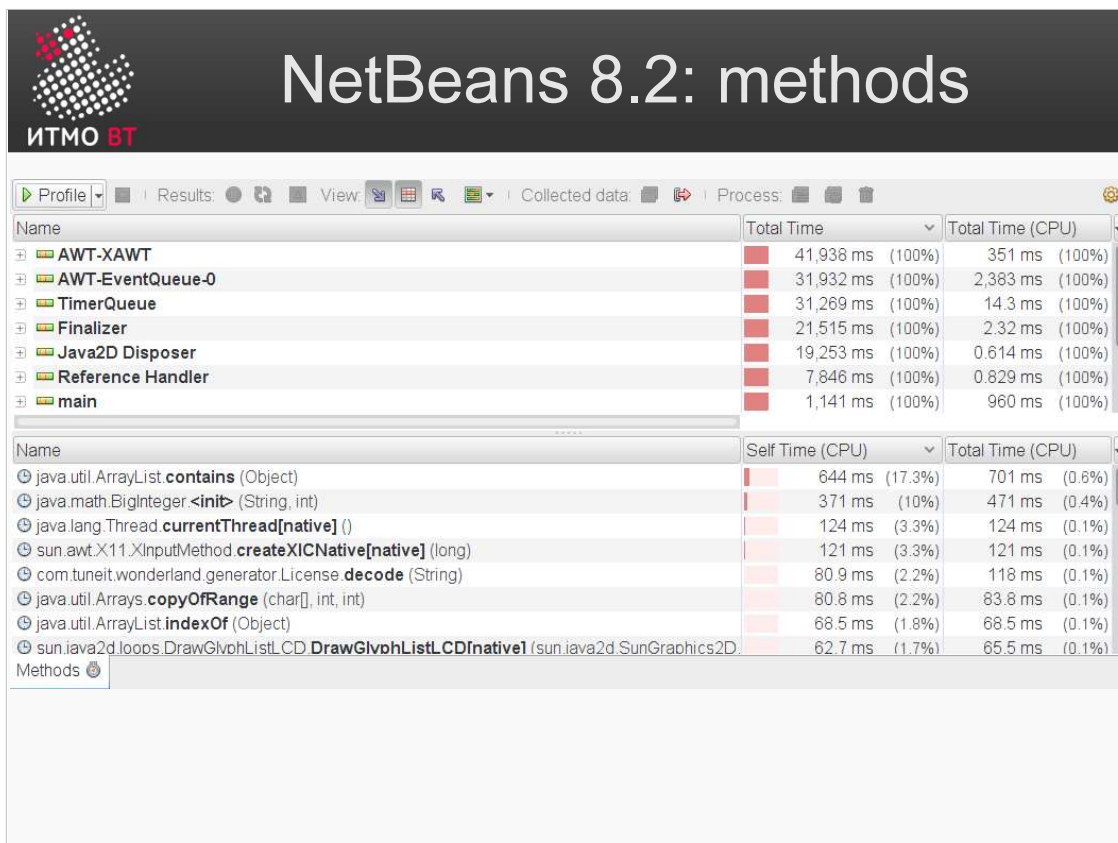
Согласно нисходящей модели поиска узких мест, после общесистемного мониторинга выявляется одно или несколько приложений, которые функционируют неудовлетворительно. Если мы являемся разработчиками этих приложений, и у нас есть исходные коды, то мы можем перейти к исследованию наших приложений для поиска алгоритмических дефектов.

Для этого существуют т.н. профилировщики приложений. С их помощью можно узнать время исполнения функции/метода, объём созданных объектов в памяти, проследить за потоками приложений и борьбой потоков за захват блокировки, временем ожидания ими освобождения блокировок (lock contentions) и др.

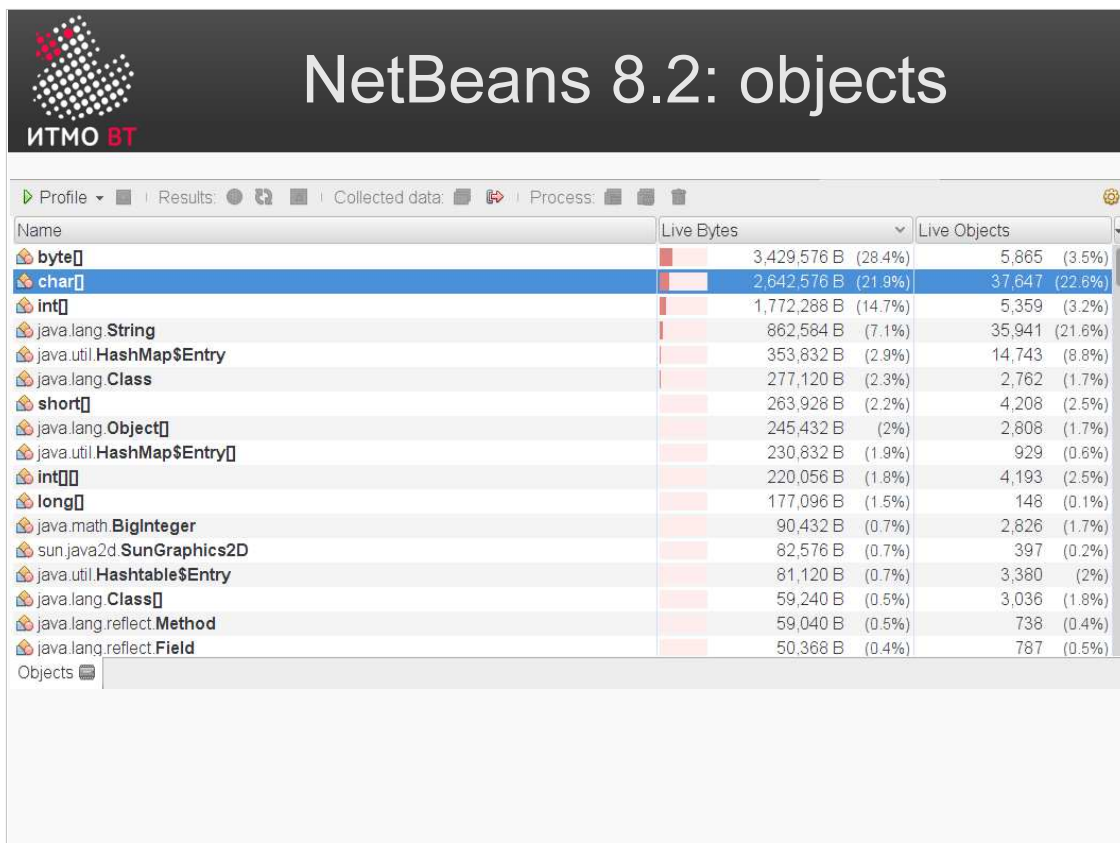
Существуют два основных подхода к профилировке. Согласно первому, диагностические точки можно внедрять в сами функции из указанного набора (например, в начале и в конце функции). Данный способ мониторинга является интрузивным, поэтому следует сначала определить проблемные места, и уже в них ставить диагностические точки.

Второй подход предполагает использование прерываний программы с заданной периодичностью. Профилировщик прерывает работающую программу и собирает интересующую его информацию. Для определения времени исполнения функций используется состояние стека программы в момент прерывания. Для определения характеристик использования памяти собирается информация о куче (heap).

Интервал прерываний выбирается так, чтобы не слишком сильно снижать производительность, и в то же время включить в выбранные данные даже методы с малой продолжительностью работы. Так работают коммерческие профилировщики, например, Performance Analyzer из Oracle Solaris Studio.



На слайде показан снимок экрана профилировщика из NetBeans 8.2. Отображены потоки в составе приложения и методы, занимающие значительное время в CPU. Можно отсортировать методы по времени исполнения, найти "горячий" метод и попытаться провести его оптимизацию.



Действия, аналогичные описанным выше, можно сделать и применительно к сбору информации о созданных в программе объектах. В данном примере больше всего места занимают байтовые массивы (byte[]). Если занимаемое место во время наблюдения растет, то имеет место так называемая *утечка памяти* — занятие и неосвобождение ресурсов. В java это означает, что где-то в программе сохраняются ссылки на созданный объект и сборщик мусора не может его уничтожить.



Trade-offs, алгоритмы и архитектура ПО

- Различные подсистемы ЭВМ взаимозависимы:
 - Чем быстрее доступ к данным, тем больше тратим памяти:
 - Linear search **vs** indexing.
 - Direct block reading from disk **vs** buffering in memory.
- Выбор алгоритма и архитектуры может ускорить приложения в миллионы раз:
 - см. курс "Алгоритмы и структуры данных".
 - Книга "PATTERN-ORIENTED SOFTWARE ARCHITECTURE: A system of pattern" Frank Buschmann and other. Siemens AG, Germany.

Борьба за производительность сопряжена с компромиссами (trade-offs). Изменения в одном месте (компоненте, методе, алгоритме и т.п.) неизбежно ведут к изменениям в других местах. Например, чем быстрее мы хотим получить доступ к данным, тем больше нам потребуется на это памяти: последовательный поиск в массиве (linear search) всегда вступает в противоречие в плане скорости/занимаемой памяти с индексированием (indexing), так как для индекса требуется дополнительная память, но его присутствие намного ускоряет поиск.

Таким образом, время, потраченное CPU, связано с тем количеством памяти, которое требует программа: если сделать требования программы скромнее, то она будет требовать больших ресурсов CPU, а программа с большими требованиями к памяти будет работать быстрее за счёт используемых алгоритмов. Однако память тоже является ограниченным ресурсом.

Другим примером компромисса является блочный доступ к диску с кэшированием блоков данных в ядре. Больше кэш — в общем случае, быстрее чтение-запись, но меньше памяти останется для других задач.

Повторимся, грамотный выбор алгоритма и учёт требований архитектуры может ускорить приложение в миллионы раз!



Рецепты: высокий %SYS

- **Высокий ввод-вывод (диск, сеть, ...):**
 - Реже писать/читать. Сжимать данные.
 - Буферизация, более быстрые устройства.
 - Размер блока передачи.
- **Планировщик — большая runqueue / CS:**
 - Обратить внимание на потоки (ПО или OS).
- **Paging/swapping:**
 - Больше памяти системе, меньше процессам.
 - Запретить выгрузку из памяти.
- **Другие системные функции ядра:**
 - Найти и попытаться исключить.
 - Настраивать ядро.

Рассмотрим, что можно изменить в ваших программах, если вы сталкиваетесь с конкретными симптомами проблем производительности вычислительной системы.

Причин, которые порождают высокую загрузенность CPU задачами уровня ядра (высокий %SYS) может быть много. Одни из самых распространённых:

1) Высокая или непродуманная нагрузка на подсистему ввода-вывода. В таком случае нужно реже писать/читать, а также стараться сжимать данные при помощи алгоритмов сжатия, или использовать бинарное представление данных числа вместо строкового. Помогают также буферизация и замена устройств на более быстрые. Можно попытаться синхронизировать размер блока передачи с ОС или устройствами хранения.

2) Недостатки в работе планировщика: чрезмерно частая диспетчеризация, приводящая к избыточному переключению контекстов процессов и т.п. В этом случае нужно проверить, не слишком ли много у вас используется потоков, и что делает в это время ОС.

3) Избыточная подкачка страниц. Если существуют проблемы с виртуальной памятью, то можно выдать больше памяти системе за счёт процессов. Другим решением может быть запрет выгрузки некоторых критически важных процессов из памяти, вследствие чего они не будут подвержены swapping'у.

4) Трата времени процессора в других системных функциях и процессах ядра. В этом случае можно попытаться найти и исключить лишние системные процессы или настроить параметры ядра. Тонкая настройка ядра связана с глубоким пониманием его структуры и анализом большого объёма документации.



Рецепты: высокий %IO wait

- Проблемы приложений:
 - Оптимизировать запросы к диску.
 - Согласовать блок приложения с ОС и дисками.
- Буфера/кэши:
 - Расширить память.
 - Настроить буферы/кэши.
- Аппаратура:
 - Купить новую дисковую подсистему.
 - Купить SSD или flash-кэш (для случайного доступа).

Указанные ниже проблемы порождают высокое время ожидания CPU (высокий %IO wait):

1) Проблемы, связанные с самими приложениями. Как и в первом решении при высоком %SYS, следует оптимизировать запросы к диску: меньше и реже читать/писать. Если предполагается активный обмен данными, нужно вести обмен большими порциями за одну операцию. Нужно также проверить, согласован ли блок приложения с ОС и дисками: например, если при размере stripe-size в 16 Кб, читать данные блоками по 8 Кб, то происходит, фактически, двойное прочтение, и преимущества RAID-массива сходят на нет.

2) Буфера/кэши, т. е., в системе выделено мало памяти под промежуточное хранение данных. В этом случае можно расширить память, либо настроить использование системных кэшей.

3) Проблемы с аппаратурой. Можно купить новую, более совершенную дисковую подсистему. Для ускорения ввода-вывода очень эффективны SSD или отдельные карты flash памяти, непосредственно устанавливаемые в шину (например PCIe) вычислительной системы. Также повторимся, что такие устройства хорошо подходят для ПО, осуществляющего случайный доступ на диск.



Рецепты: высокий %Idle

- Проблемы приложений — мало RUNNABLE:
 - Параллелить всё, что можно.
 - Добавить потоки в пулы приложений.
 - Анализ и оптимизация блокировок.
 - Lock-free алгоритмы.
- Проблемы ОС:
 - Блокировки — можно мониторить через средства ОС.
 - Настройка параметров подсистем ядра.
 - Поиск дефектов в багтрекере.

Перечисленные на слайде проблемы порождают высокое время простоя CPU (высокое %Idle):

1) В системе мало процессов в стадии выполнения.

В этом случае может помочь распараллеливание алгоритмов в приложении, особенно, если система многопроцессорная. Если существуют пулы потоков-worker'ов, то в них можно добавить дополнительные потоки для равномерной загрузки всех ядер ЦПУ.

Вторым источником проблем в данном случае могут являться внутренние блокировки в приложении, например, когда много потоков пытаются завладеть одним и тем же участком кода. В этом случае блокировки необходимо оптимизировать. Основной рецепт — держать блокировку как можно меньше времени, а также по возможности использовать более «лёгкие» типы блокировок.

Кроме этого, возможно использование алгоритмов без блокировок (Lock-free).

2) Проблемы могут также корениться в самой ОС.

В большинстве случаев это связано с дефектами в ОС на системных блокировках. Следует посмотреть багтрекер ОС и сопоставить написанное там с симптомами конкретной проблемы. Может помочь также настройка параметров подсистем ядра.



Рецепты: высокий %user

• Проблемы приложений:

- Алгоритмы с меньшей алгоритмической сложностью.
- Повторное использование объектов.
- Избавиться от активных опросов и ожиданий в циклах.
- Искать промахи мимо кэшей и TLB.
- Учитывать размер и организацию кэшей:
 - Размер кэшей ограничен.
 - Паддинг и разрывание объектов.
 - Использовать более слабые примитивы синхронизации.
- "Биндить" процессы/потоки к процессорам.
- Оптимизировать на уровне ассемблера/компилятора.
- Повысить частоту процессора, увеличить количество ядер, использовать GPU/криптопроцессоры.

Часто можно встретить ошибочное мнение, что если процессор на 100% загружен, то он эффективно работает, и сделать ничего нельзя. Как в таком случае еще сильнее ускорить работу? Эта задача обычно сложнее (и интереснее) предыдущих.

Для определения узкого места в программе необходимо воспользоваться средствами профилирования. Они помогут найти наиболее затратные функции, объекты с частым созданием/удалением в памяти и другие аномалии вашего ПО. Наиболее действенные рецепты следующие:

1) Использование алгоритмов с меньшей алгоритмической сложностью (помним про баланс и компромиссы — если где-то что-то убыло, значит где-то что-то прибыло).

2) Принцип повторного использования объектов. Крупные объекты и структуры нет необходимости удалять и создавать повторно. Можно взять старый объект и установить в него новые данные.

Этот принцип, например, применяется в UNIX для структур процессов. Когда процесс "умирает" (выходит по функции `exit()` или `kill()`), он сначала попадает в состояние "зомби". Когда код возврата прочитан родительским процессом, он должен окончательно "умереть" и исчезнуть из структур ядра. Однако, ядро складывает часть "зомби" на "кладбище" (death row), и когда необходимо создать новый процесс, он "восстаёт из мертвых" — его структура повторно используется.

3) На уровне микроархитектуры важно избавляться от кэш-промахов и промахов мимо TLB (translation lookaside buffer, буфер ассоциативной трансляции). Кэш-промахи можно исправить работой над структурами данных (группировка данных для попадания в одну строку кэша, паддинг и др.) Промахи мимо TLB можно исправить использованием больших страниц памяти.

Для исключения "остывания кэшей" можно закреплять потоки за процессорами. ("биндинг" или установка аффинити). Кроме того, биндинг позволит обращаться к локальной памяти процессора при NUMA архитектуре, минимизируя запросы через шину к другой процессорной плате.

Для истинных гиков можно попробовать переписывать части кода на ассемблере и использовать специальные средства аппаратуры — GPU и криптопроцессоры.



Счётчики производительности CPU

- Аппаратные регистры внутри процессора:
 - Позволяют накапливать произошедшие события.
 - Количество регистров и набор событий сильно зависит от процессора:
 - Обычно всегда присутствуют CPU+MMU+TLB.
- Поддерживаются специальными средствами.
 - Linux perf:
 - `perf list | grep -i hardware`
 - Коммерческие средства разработки.



Для анализа микроархитектуры внутри процессора существуют т.н. счётчики производительности ядра. Это регистры, собирающие данные тех параметров, которые им назначаются (не определенные заранее). Данные регистры являются счётчиками, которые наращиваются самим процессором. Число таких счётчиков ограничено (обычно не более 2-4), поэтому одновременно можно наблюдать только за несколькими параметрами.

В Linux счётчиками управляет команда `perf`. Используются также коммерческие средства разработки.

Для примера приведём список событий, по которым `perf` может собирать информацию:

branch-instructions branch-misses cache-misses cache-references cpu-cycles instructions ref-cycles stalled-cycles-backend stalled-cycles-frontend	L1-dcache-load-misses L1-dcache-loads L1-dcache-prefetch-misses L1-dcache-prefetches L1-dcache-store-misses L1-dcache-stores L1-icache-load-misses L1-icache-loads LLC-load-misses LLC-loads LLC-prefetch-misses LLC-prefetches LLC-store-misses LLC-stores	branch-load-misses branch-loads dTLB-load-misses dTLB-loads dTLB-store-misses dTLB-stores iTLB-load-misses iTLB-loads node-load-misses node-loads node-prefetch-misses node-prefetches node-store-misses node-stores
---	--	---